RELATÓRIO TÉCNICO — PROJETO INTEGRADOR FULL STACK
Modernização de Sistema Legado: Gestão e Análise de Marketing Digital

Instituição: ESEG - Escola Superior de Engenharia e Gestão
Curso: Engenharia de Computação - Programação Web
Estudante: Eduardo Marques Fadini - 42721

1. Inventário e Diagnóstico da Base de Dados Legada

A base de dados legada que estou usando (18_marketing_digital.xlsx) traz um conjunto de planilhas desnormalizadas com dados simulados de marketing digital do período de 01/01/2024 a 31/12/2025

O arquivo original é dividido basicamente em duas partes: 8 abas com os dados operacionais e analíticos (que cobrem contas, campanhas, anúncios, públicos, desempenho diário, funil mensal, e-mail marketing e landing pages) e outras 2 abas que servem apenas para documentação, que são dicionário de dados e os metadados.

1.1. Problemas Estruturais Identificados no Legado

Quando fiz a análise crítica da estrutura original, deu para notar seis falhas principais que complicam bastante a integridade, a manutenção e a confiabilidade dos dados.

A primeira e mais pesada é a Redundância e Desnormalização Severa. Olha só como isso se espalha pelo arquivo:

- Na tabela desempenho_diario: Ela duplica chaves estrangeiras transitivas (id_conta, id_campanha) e copia várias colunas descritivas que já deveriam estar em outras tabelas. É o caso de canal e objetivo (que são de campanhas), formato (de anúncios) e todo o bloco de faixa_etaria, genero, cidade, estado e regiao (que pertencem a públicos).
- Na tabela funil_mensal: O atributo canal se repete sem necessidade, já que a tabela já tem a chave id_campanha.
- Na tabela landing_pages: A coluna produto aparece de novo, mesmo o produto já estando associado diretamente à campanha.
- Impacto: O risco de dar ruim na hora de atualizar os dados (anomalias de atualização) é gigante. Se a gente precisar corrigir o canal de uma campanha ou a localização de um público-alvo na tabela de origem, os milhares de registros diários e mensais vão acabar ficando desatualizados e inconsistentes entre si.

O segundo problema sério que encontramos foi a Persistência de Colunas Calculadas e a Imprecisão Numérica:

- Métricas salvas como colunas fixas: A base legada armazena direto no banco várias métricas que deveriam ser calculadas na hora da consulta. É o caso de ctr, cpc, taxa_conversao e roas na tabela desempenho_diario; taxa_clique, taxa_lead e taxa_venda_por_lead na funil_mensal; e taxa_conversao na landing_pages.
- Erros de arredondamento: Como usaram ponto flutuante sem nenhum tratamento, a planilha original está cheia de dízimas e erros de precisão bizarrinhos. Por exemplo,

tem ctr valendo 0.008800000000000001 no registro id_desempenho = 1 e receita_atribuida marcando 8611.610000000001 no registro id_email = 2.
- Impacto: Se a gente atualizar qualquer dado de origem, como o número de cliques, impressões, custo ou receita, todos esses indicadores salvos vão ficar errados e defasados automaticamente, porque eles não recalculam sozinhos.

O terceiro ponto crítico é a Fragmentação de Datas:

- Datas divididas em várias colunas: As tabelas desempenho_diario, funil_mensal e email_marketing quebram a informação de tempo em várias colunas de números inteiros (ano, mes, trimestre).
- Formato bruto do Excel: Para piorar, elas guardam a data principal naquele formato de número de série do Excel (tipo o 45292 que aparece lá), em vez de usar um tipo de dado de data de verdade (DATE).
- Impacto: Além de deixar as consultas por intervalo de tempo muito mais difíceis e pesadas de rodar no banco de dados, essa divisão abre margem para erros bobos de digitação. Por exemplo, um registro pode ter o campo trimestre totalmente incompatível com o mes informado na mesma linha.

O quarto problema é a Despadronização dos Campos de Situação (Status):

- Falta de padrão entre as tabelas: Cada tabela define os seus status de um jeito totalmente isolado.
- Mistura de gêneros e sinônimos: Eles misturaram gêneros gramaticais e termos que significam quase a mesma coisa, sem nenhum controle de transição. Olha a bagunça nos campos:
    - contas.status_conta: usa no feminino, 'Ativa' e 'Inativa'.
    - campanhas.status_campanha: usa no feminino, 'Ativa', 'Pausada' e 'Encerrada'.
    - anuncios.status_anuncio: muda para o masculino, 'Ativo', 'Pausado' e 'Finalizado'.
    - landing_pages.status_landing_page: volta para o feminino, 'Ativa', 'Pausada' e 'Arquivada'.
- Impacto: Fazer qualquer query ou filtro global usando o status vira um pesadelo. Além disso, sem uma regra clara ou uma tabela de transição de status, o sistema aceita qualquer palavra, o que quebra a consistência do negócio.

O quinto ponto é a Repetição de Atributos de Localização:

- Dados geográficos espalhados: Os campos cidade, estado e regiao aparecem de forma idêntica nas tabelas contas, publicos e desempenho_diario.
- Impacto: Além de gastar espaço de armazenamento à toa repetindo o nome de cidades e estados milhares de vezes, isso quebra o conceito de herança de dados. A localização deveria estar amarrada apenas à conta ou ao público-alvo de origem. Do jeito que está, se houver um erro de digitação em uma linha da tabela desempenho_diario, a gente perde o rastreio correto de onde aquele resultado veio.

O sexto ponto da análise é a Inexistência de Autenticação, Autorização e Integridade Referencial:

- Sem controle de acesso: O sistema legado não tem nenhum tipo de controle de perfil de usuário (quem pode ver ou editar o quê).
- Falta de travas no banco de dados: Não existem restrições de chave estrangeira (FOREIGN KEY) nem validações de domínio (regras de CHECK ou campos obrigatórios com NOT NULL).
- Impacto: O banco aceita literalmente qualquer coisa. Sem essas travas de segurança, o sistema deixa passar bizarrices como custos com valores negativos ou campanhas com a data de término cadastrada para antes da data de início. Fica impossível garantir que a base de dados seja confiável.

2. Soluções Adotadas e Justificativas Técnicas

Modelagem do Banco e Normalização (3FN):

- Problema: As tabelas DesempenhoDiario e ResumoFunil estavam cheias de redundância e chaves transitivas.
- Solução: Normalizei tudo até a 3ª Forma Normal (3FN). Agora, DesempenhoDiario só tem relacionamento direto (@ManyToOne) com Anuncio e PublicoAlvo. Se a gente precisar de dados da campanha, do canal ou da conta, pegamos direto pela navegação do grafo de objetos em Java (ex: desempenho.getAnuncio().getCampanha().getConta()), garantindo a integridade dos dados.

Reaproveitamento de Código com @MappedSuperclass:

- Problema: ResumoFunil e DesempenhoDiario compartilham exatamente os mesmos atributos de métricas (impressões, cliques, leads, custo, receita, etc.). Criar as mesmas colunas nas duas entidades seria vacilo e geraria código duplicado.
- Solução: Criei uma superclasse abstrata chamada MetricasFunil e coloquei um @MappedSuperclass nela. Assim, centralizamos os atributos e os métodos de cálculo em um lugar só. O banco não cria uma tabela física para a superclasse, o que é ótimo para a performance de leitura, já que o sistema processa 24.910 registros diários.

Encapsulamento de Regras de Negócio e Uso de BigDecimal

- Problema: O sistema antigo usava double para dinheiro e taxas, o que dava pau de arredondamento financeiro, além de persistir colunas calculadas no banco sem necessidade.
- Solução: Arrancamos as colunas calculadas do banco. Agora, os cálculos rodam em métodos de domínio direto nas entidades:
    - calcularCTR() e calcularROAS() (em MetricasFunil)
    - calcularTaxaConversao() (em LandingPage)
    - calcularTaxaAbertura() (em EmailMarketing)

Padrão Value Object para Endereços

- Problema: Conta e PublicoAlvo precisavam armazenar dados de localização (cidade, estado e região), gerando repetição de atributos parecidos em tabelas diferentes.
- Solução: Isolamos esses campos em uma classe Localizacao anotada com @Embeddable. Daí, usamos composição nas entidades principais com @Embedded, aplicando o conceito de Value Object (VO) do DDD.

Máquina de Estados com Enum

- Problema: O controle de status das campanhas era bagunçado e permitia transições bizarras no banco.
- Solução: Criamos um Enum Status com os estados fixos (ATIVO, PAUSADO, ENCERRADO, INATIVO e ARQUIVADO). Dentro da entidade Campanha, criamos o método alterarStatus(Status novoStatus) que funciona como uma máquina de estados real, validando o fluxo (tipo impedir que uma campanha ENCERRADO seja reativada).

Segurança e Controle de Acesso (RBAC)

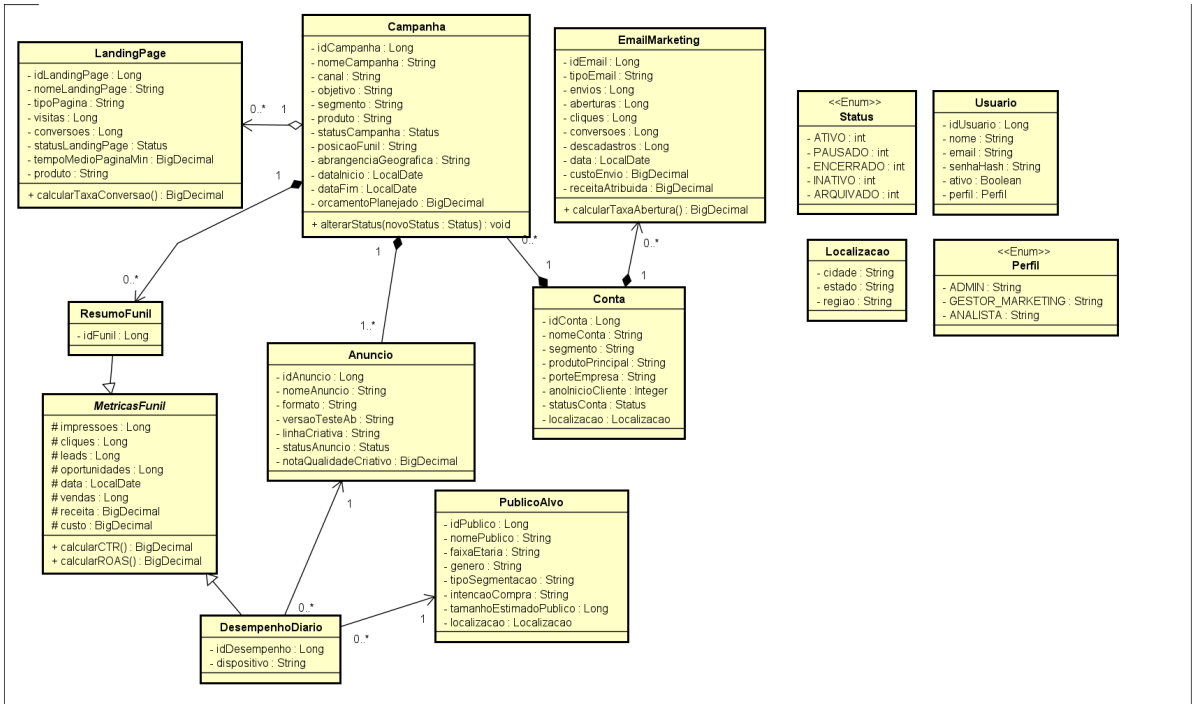
- Solução: Modelamos a entidade Usuario integrada ao Spring Security. As senhas são salvas no banco criptografadas usando BCrypt (senhaHash). O controle de autorização nas rotas e telas do Thymeleaf é feito por perfis de acesso (Roles) baseados no Enum Perfil (ADMIN, GESTOR_MARKETING e ANALISTA).

3. Matriz de Rastreabilidade de Requisitos

ID	Requisito Original (Legado)	Problema/ Lacuna	Requisito Revisado ou Novo	Justificativa	Implementação (Spring / JPA / Thymeleaf)	Teste / Evidência
RF01	Registros de Contas, Campanhas e Anúncios mantidos em abas separadas sem vínculo forçado.	Falta de integridade referencial; possibilidade de criar anúncios para campanhas inexistentes.	[Revisado] CRUD completo de Campanha e Anuncio com integridade relacional obrigatória às entidades Conta e PublicoAlvo.	Garante consistência dos dados persistentes e navegação segura entre entidades.	Entidades JPA (@ManyToOne, @OneToMany), Repositories, Controllers MVC e formulários Thymeleaf com Bootstrap.	Teste de persistência validando criação/edição e bloqueio de exclusão de registos pais com dependências.
RF02	Campos de status em texto livre (Ativa, Ativo, Encerrada, Finalizado).	Nomenclaturas divergentes entre abas e ausência de regra de transição de estados.	[Revisado] Padronização via enum Status e validação de transições válidas no método alterarStatus().	Evita estados ilegais no ciclo de vida da campanha (ex: ENCERRADO voltar para ATIVO).	Enum Status mapeado com @Enumerated(EnumType.STRING) e validação de negócio na entidade/serviço.	Teste unitário JUnit comprovando o bloqueio de transição inválida com lançamento de exceção
RF03	Colunas fixas para ctr, roas, taxa_conversao e decomposição manual de datas (ano, mes, trimestre).	Erros de precisão decimal e inconsistência quando métricas brutas são atualizadas.	[Revisado] Cálculo dinâmico de indicadores em tempo de execução com BigDecimal e uso de LocalDate.	Elimina redundância de colunas calculadas (3FN) e assegura exatidão matemática.	Métodos calcularCTR(), calcularROAS(), calcularTaxaConversao() e calcularTaxaAbertura() nas classes de	Teste unitário validando o cálculo exato e o tratamento de divisão por zero quando impressoes = 0.

					domínio.	
RF04	Registo de desempenho_diario repetindo 10 colunas de campanha, conta e público.	Redundância em 24.910 linhas e alto risco de divergência cadastral.	[Revisado] Normalização de DesempenhoDiario mantendo apenas o vínculo relacional com Anuncio e PublicoAlvo.	Aplica a 3ª Forma Normal (3FN), reduzindo o tamanho da base e garantindo fonte única da verdade.	Classe DesempenhoDiario extends MetricasFunil ligada a Anuncio e PublicoAlvo.	Consulta JPA com JOIN exibindo o relatório diário completo na interface Thymeleaf sem colunas duplicadas.
RF05	(Inexistente no legado) Acompanhamento do limite de orçamento da campanha.	O legado guarda orcamento_planejado, mas não alerta quando o custo real supera esse valor.	[Novo] Alerta de estouro de orçamento e bloqueio de ativação de novos anúncios quando o custo acumulado atingir o orcamento Planejado.	Agrega controle financeiro real ao processo de gestão de campanhas.	Regra na camada Service somando os custos da campanha + alerta visual Bootstrap no painel Thymeleaf.	Teste automatizado simulando campanha com custo acima do planejado e verificando o bloqueio.
RF06	(Inexistente no legado) Controle de acesso por perfil de utilizador.	Qualquer pessoa podia editar dados financeiros ou apagar campanhas na folha de cálculo.	[Novo] Autenticação e autorização com perfis distintos (ADMIN, GESTOR_MARKETING, ANALISTA).	Protege operações sensíveis no servidor (ex: ANALISTA apenas consulta; GESTOR_MARKETING edita campanhas; ADMIN gere contas/utilizadores).	Spring Security, BCryptPasswordEncoder, controle de rotas e anotações de segurança nos Controllers.	Teste de integração HTTP comprovando retorno 403 Forbidden para utilizador sem permissão.

4. Modelagem Orientada a Objetos - Diagrama de Classes

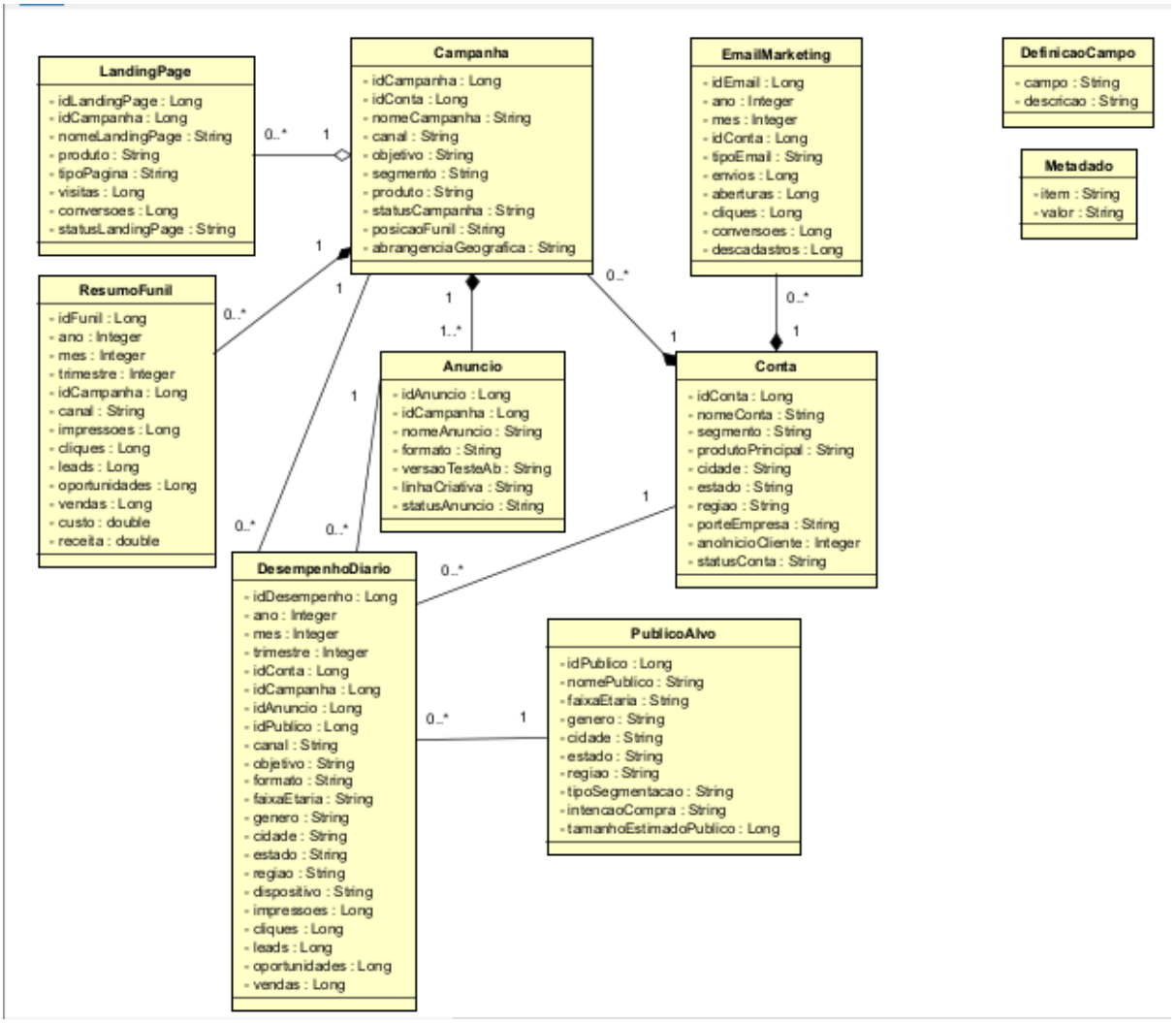


Resumo das Responsabilidades das Classes:

- Conta: Representa o cliente anunciante, agregando a sua Localizacao e compondo o ciclo de vida das suas Campanhas e disparos de EmailMarketing.
- Campanha: Entidade central do domínio; gere o orçamento planejado, período de vigência (dataInicio, dataFim), regras de mudança de Status (alterarStatus) e compõe os seus Anuncios, ResumoFunil e LandingPages.
- Anuncio: Representa a peça criativa vinculada a uma campanha, incluindo formato, versão de teste A/B e nota de qualidade do criativo.
- PublicoAlvo: Define o perfil demográfico, comportamental e geográfico (Localizacao) utilizado na segmentação dos anúncios.
- MetricasFunil (Abstrata): Superclasse que centraliza os indicadores quantitativos de funil (impressoes, cliques, leads, oportunidades, vendas, data, receita, custo) e os cálculos de CTR e ROAS.
- ResumoFunil e DesempenhoDiario: Especializações de MetricasFunil para consolidação por campanha e por anúncio/público/dispositivo, respetivamente.
- LandingPage e EmailMarketing: Entidades de suporte à conversão e relacionamento, dotadas de métodos próprios para cálculo de taxa de conversão e taxa de abertura.
- Usuario e Perfil: Responsáveis pela autenticação e controle de acesso da aplicação web.

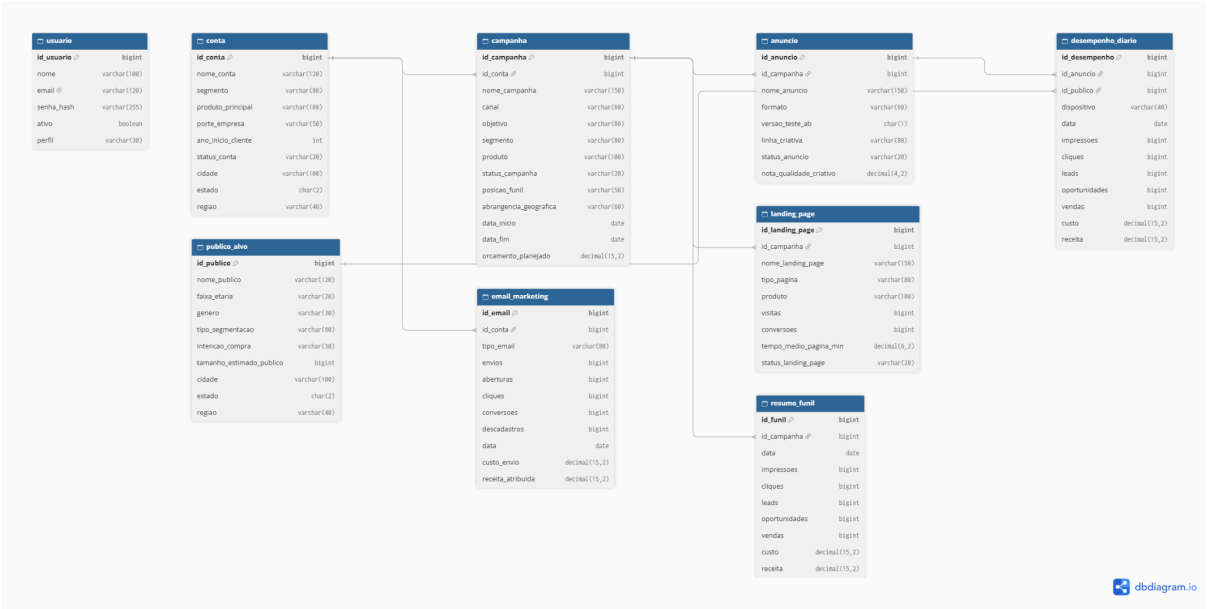
5. Evolução do Modelo Relacional: Legado vs. Evoluído

5.1. Estrutura Original — Modelo Legado (Antes)



Modelo Legado (Antes)

5.2. Estrutura Proposta — Modelo Evoluído e Script DDL (Depois)



5.3. Comparativo das Tabelas (Antes vs. Depois)

Tabela no Legado (Excel)	Problemas no Legado	Tabela no Modelo Evoluído (SQL/JPA)	Melhorias Aplicadas na Evolução
contas	Status em texto livre (Ativa/Inativa); sem chave primária formal.	conta	PRIMARY KEY (id_conta), validação CHECK no status (ATIVO/INATIVO) e colunas de Localizacao embutidas (@Embedded).
campanhas	Datas em formato numérico do Excel; status despadronizado; sem FK ativa.	campanha	FOREIGN KEY (id_conta), datas como DATE nativo, restrição CHECK (data_fim >= data_inicio) e orcamento_planejado >= 0.
anuncios	Status 'Finalizado' divergente de 'Encerrada'; sem FK ativa.	anuncio	FOREIGN KEY (id_campanha), status padronizado (ENCERRADO) e restrição CHECK (nota_qualidade_criativo BETWEEN 0

			AND 10).
publicos	Sem restrições de domínio ou tipos numéricos validados.	publico_alvo	PRIMARY KEY (id_publico), Localizacao embutida e CHECK (tamanho_estimado_publico > 0).
desempenho_diario	29 colunas (10 colunas copiadas de outras tabelas, 4 taxas calculadas fixas e 3 colunas de data repetidas).	desempenho_diario	Reduzida para 12 colunas essenciais (3FN): chaves id_anuncio e id_publico, dispositivo e métricas herdadas de MetricasFunil.
funil_mensal	Coluna canal duplicada; 3 taxas calculadas salvas; data dividida em ano, mes, trimestre.	resumo_funil	Vinculada via FOREIGN KEY (id_campanha), data unificada em DATE e taxas calculadas na camada de domínio.
email_marketing	Data dividida em data_envio, ano, mes; erros de dízima em receita_atribuida.	email_marketing	FOREIGN KEY (id_conta), tipo DECIMAL(15,2) exato para custos/receitas e DATE unificado.
landing_pages	Coluna taxa_conversao persistida; status 'Arquivada'.	landing_page	FOREIGN KEY (id_campanha), cálculo dinâmico de conversão e status padronizado (ARQUIVADO).
(Inexistente)	Sem controle de utilizadores.	usuario	Nova tabela com email UNIQUE, senha_hash e perfil validado (ADMIN, GESTOR_MARKETING, ANALISTA).

6. Estratégia de Migração e Limitações Conhecidas

- Estratégia de Migração dos Dados Legados: Na transição para o banco de dados relacional do projeto Spring Boot, os dados das 8 abas operacionais da folha de cálculo 18_marketing_digital.xlsx serão higienizados e importados respeitando a

ordem de integridade referencial: 1º conta e publico_alvo → 2º campanha e email_marketing → 3º anuncio, landing_page e resumo_funil → 4º desempenho_diario. Durante a carga, os valores de status femininos/sinónimos ('Ativa', 'Finalizado') serão convertidos para as constantes do enum Status (ATIVO, ENCERRADO) e as datas seriais do Excel serão convertidas para o formato ISO (YYYY-MM-DD).

- Limitações Conhecidas e Melhorias Futuras: Na tabela original funil_mensal, os registros possuem apenas ano e mes (sem o dia específico); na migração para o atributo data : LocalDate de ResumoFunil, será convencionado o primeiro dia de cada mês (YYYY-MM-01) como data de referência. Como melhoria futura, o resumo mensal poderá ser gerado integralmente via Database View ou consulta agregada (GROUP BY) a partir de desempenho_diario.